# CS 2160: Computer Organization & Assembly Language Programming

Instructor: Himon Thakur

Fall 2026

## Project #01

- This project is worth **100 points**. You are expected to submit a Project Report and relevant screenshots for each deliverables.
- Please type your project report and format it properly (the TA will thank you)
- **10% bonus points** will be available for anyone submitting LaTeX-formatted homework
- Please use the internet and all other resources to help you with the assignment, not write it for you
- You should complete this project individually. However, feel to discuss ideas with other humans or non-human entities if you need help. The final deliverables, however, should be completed and submitted by you. Please cite any outside sources in the report.
- START EARLY!

## The QT RV Simulator

This project is intended to familiarize you with the **RISC-V simulator** we will be using in this class (`https://github.com/cvut/qtrvsim` or `https://comparch.edu.cvut.cz/qtrvsim/app/`). The latter is a web-based tool, so you can work directly through the browser. Here are some tips for using the simulator:

- Save your programs early and often. Consider using a version control software like git to ensure you can easily return to previous states of your code.
- On some systems the simulator can encounter performance issues. If you encounter this, try switching browsers and/or operating systems.
- Although the simulation features are excellent, it's text editing ability leaves something to be desired. Consider writing your code on your preferred IDE/text editor, and importing to the simulator when ready to run. Feel free to use *Visual Studio Code* with the **RISC-V** extension, which provides some nice amenities like syntax highlighting and indentation.

You may format your code like this.

```
// Layout of source code

// Start of Instructions section

. text
// some instructions


// End of Instructions and start of Data section
. data
// place some values here
```

More information on how to use the simulator can be found in **Lecture Slide 6**.

## Deliverable 01: Arithmetic ($10 + 15 = 25$ pts)

Calculate the *multiplication* of **two integer** numbers by writing a program in the simulator. Store the *operands* into registers and perform operations on it. Some comments/hints:

1. You can use *load immediate* instruction `li` to initialize data into a register.
2. In your report, include screenshot of i) your program code, and, ii) the **Registers** window that shows the correct value of the output register.

## Deliverables 02: Data Manipulation ($25 \times 3 = 75$ pts)

The snippet below shows one way to create arrays of *32-bit* elements in the data section.

```
.data
A: .word 2, 3, 7 , y , 0x1A5
B: .word 6, 5, 4
```

The `A:` label makes space in memory for the five specified consecutive words. The `B:` label makes space in memory immediately after A for the three specified consecutive words.

Start a new source in the simulator, and initialize it with the program

```
.text

.data
A: .word 2, 3, 7 , B , 0x1A5
B: .word 6, 5, 4
```

It has an empty instructions section, and its data section is the one shown above. Then, complete the following tasks.

1. (5 pts) Open the Memory window, and take a screenshot that clearly shows all elements of the arrays **A** and **B**.

2. (5 pts) Report the base address of **A**, the base address of **B**, and the value word stored in `&A[3]`.

3. (10 pts) Now, fill in the .text section with instructions that will perform the addition `A[1] + B[2]`. Store the result in register `a0`. Take a screenshot of both your program and the registers window that shows the correct value inside `a0`. Be sure to follow good practice and properly terminate the program.